Cardiovascular Disease Risk Classification Pipeline

Capstone Project Report – AI & Data Science

Assadiki Zaid & Eddhouir Mohammed
Academic Mentor: Dr. Rym Nassih

Project Repository: `GitHub Link`

June 18, 2026

Contents

1 Executive Summary

Early detection of cardiovascular complications can drastically alter patient outcomes and ease the critical burden on healthcare infrastructure. This project focuses on building a predictive pipeline to determine the presence or risk level of cardiovascular disease based on clinical measurements, lifestyle habits, and patient biometric data. By analyzing heterogeneous health indicators, the machine learning model serves as a clinical decision-support tool designed for medical professionals and healthcare providers. It enables them to identify high-risk individuals accurately, prioritize preventative therapeutic interventions, and optimize patient monitoring strategies before severe clinical events occur.

2 Phase 1: Problem Framing, Dataset Description, & AI Context

The objective of this project is to develop a machine learning pipeline that categorizes patients into high-risk (cardio=1) or low-risk (cardio=0) based on routine clinical vitals, body measurements, and lifestyle surveys. This problem is formulated as a binary classification task.

2.1 AI Context Note

In early AI, the dominant paradigm was Symbolic AI and Rule-Based Expert Systems, such as the MYCIN system developed in the 1970s. These systems relied on hand-crafted rules created by clinical experts:

$$\text{IF } systolic\_bp > 140 \text{ AND } cholesterol = 3 \text{ THEN } cardio\_risk = \text{HIGH} \tag{1}$$

While MYCIN and other expert systems were highly transparent, they suffered from the knowledge acquisition bottleneck, fragility, and an inability to learn from raw empirical data or manage high-dimensional probabilistic boundaries. The modern paradigm shift to empirical machine learning (connectionism) resolves this. Instead of manual rules, models like XGBoost learn associations directly from a dataset of 70,000 real-world observations, mapping continuous risk boundaries that are more robust to noisy data.

2.2 Repository Initialization & Structure

The complete, reproducible codebase for this project is hosted publicly on GitHub at:
`https://github.com/AssadikiZaid/ml-capstone-cardio-risk-classification-ehtp`

The repository is strictly structured to standard data science conventions:

- `/data`: Contains the raw CSV datasets and the processed SQLite database.
- `/notebooks`: Contains chronologically ordered Jupyter notebooks for EDA, Feature Engineering, and Model Training.
- `/src`: Contains reusable programmatic scripts, such as data ingestion files.
- `/models`: Stores the serialized best-performing model (`.joblib`).

2.3 Ingestion Code (src/ingest.py)

Below is the complete, comment-free code of the data ingestion script:

```python
import os
import pandas as pd

def load_and_describe_data():
    file_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'data', 'cardio_train.csv')
    df = pd.read_csv(file_path, sep=';')

    print("Shape:", df.shape)
    print("\nData Types:")
    print(df.dtypes)
    print("\nNull Counts:")
    print(df.isnull().sum())
    print("\nSummary Statistics:")
    print(df.describe())

    return df

if __name__ == '__main__':
    load_and_describe_data()
```

2.3.1 Code Dive

- `import os`: Imports the operating system module to resolve file paths.
- `import pandas as pd`: Imports the Pandas library for data manipulation.
- `file_path = ...`: Programmatically constructs the absolute path to the dataset folder.
- `df = pd.read_csv(file_path, sep=';')`: Loads the CSV file, setting the separator to a semicolon.
- `print("Shape:", df.shape)`: Outputs the number of rows (70,000) and columns (13).
- `print(df.dtypes)`: Logs the data types of all columns.
- `print(df.isnull().sum())`: Verifies that all null counts are 0.
- `print(df.describe())`: Outputs basic summary statistics.

3 Phase 2: Data Audit, Outlier Treatment, & Encoding

A comprehensive audit was performed on the raw Kaggle dataset.

3.1 Data Quality Audit

Table 1: Data Quality Audit

Variable	Type	Range	Audit Findings
age	Int (days)	10,798 – 23,713	Stored in days; requires conversion to years
gender	Cat (1/2)	1 – 2	Re-encoded to binary (0/1)
height	Int (cm)	55 – 250	Contains unphysical height values
weight	Float (kg)	10 – 200	Contains unphysical weight values
ap_hi	Int (mmHg)	-150 – 16,000	Severe outliers (logging errors)
ap_lo	Int (mmHg)	-70 – 11,000	Severe outliers (logging errors)
cardio	Binary (0/1)	0 – 1	Target variable; perfectly balanced

3.2 Outlier Detection Code

Below is the code to identify outliers using IQR and Z-score methods:

```
import pandas as pd
import numpy as np
from scipy import stats

df = pd.read_csv('../data/cardio_train.csv', sep=';')

q1_hi = df['ap_hi'].quantile(0.25)
q3_hi = df['ap_hi'].quantile(0.75)
iqr_hi = q3_hi - q1_hi
lower_hi = q1_hi - 1.5 * iqr_hi
upper_hi = q3_hi + 1.5 * iqr_hi

q1_lo = df['ap_lo'].quantile(0.25)
q3_lo = df['ap_lo'].quantile(0.75)
iqr_lo = q3_lo - q1_lo
lower_lo = q1_lo - 1.5 * iqr_lo
upper_lo = q3_lo + 1.5 * iqr_lo

outliers_iqr = df[
    (df['ap_hi'] < lower_hi) | (df['ap_hi'] > upper_hi) |
    (df['ap_lo'] < lower_lo) | (df['ap_lo'] > upper_lo)
]
print("IQR Outliers Count:", len(outliers_iqr))

z_hi = np.abs(stats.zscore(df['ap_hi']))
z_lo = np.abs(stats.zscore(df['ap_lo']))
outliers_z = df[(z_hi > 3) | (z_lo > 3)]
print("Z-score Outliers Count:", len(outliers_z))
```

3.2.1 Method Comparison

- **IQR Method:** Flagged 1,439 outliers.
- **Z-score Method ($|Z| > 3$):** Flagged 983 outliers.
- **Analysis:** The Z-score method was heavily distorted by extreme values (e.g., blood pressure of 16,000 mmHg), which artificially inflated the standard deviation. A strict physical biomedical threshold was applied to clean the cohort, removing 6,058 records and leaving a clean sample of 63,942 patients.

3.3 Exploratory Data Analysis (EDA) Summary

During our visual analysis across the 8 generated distributions and feature relationships, a critical and surprising insight emerged. While we initially expected raw Systolic Blood Pressure (*ap_hi*) to be the ultimate predictor, the EDA showed that our newly engineered feature, **Pulse Pressure** (*ap_hi - ap_lo*), actually creates a much clearer decision boundary between healthy and high-risk patients, especially for those under the age of 55.

3.4 Categorical Encoding Code

Ordinal features *cholesterol* and *gluc* were one-hot encoded:

```
df_clean = df[
    (df['ap_hi'] >= 80) & (df['ap_hi'] <= 220) &
    (df['ap_lo'] >= 50) & (df['ap_lo'] <= 140) &
    (df['ap_hi'] > df['ap_lo']) &
    (df['height'] >= 130) & (df['height'] <= 220) &
    (df['weight'] >= 40) & (df['weight'] <= 180)
].copy()

df_clean['gender'] = df_clean['gender'] - 1
df_encoded = pd.get_dummies(df_clean, columns=['cholesterol', 'gluc'], drop_first=True)
```

4 Phase 3: Feature Engineering & Data Management

4.1 Feature Engineering Log

We engineered three new clinical indicators to enrich our feature space.

Table 2: Feature Engineering Log

Feature Name	Formula	Motivation	Impact
`bmi`	weight / $(\text{height}/100)^2$	Standard clinical metric for obesity.	Improved recall on younger demographics.
`pulse_pressure`	ap_hi - ap_lo	Indicator of arterial stiffness and vessel health.	Top 3 ranked feature by Mutual Information.
`age_chol_risk`	$(\text{age}/365.25) \times \text{chol}$	Captures compounding risk of aging with high cholesterol.	Slightly improved tree model precision.

4.2 Feature Engineering & Ranking Code

```
import pandas as pd
import numpy as np
from sklearn.feature_selection import mutual_info_classif

df_clean['bmi'] = df_clean['weight'] / ((df_clean['height'] / 100) ** 2)
df_clean['pulse_pressure'] = df_clean['ap_hi'] - df_clean['ap_lo']
df_clean['age_cholesterol_risk'] = (df_clean['age'] / 365.25) * df_clean['cholesterol']

X = df_clean.drop(columns=['id', 'cardio'])
y = df_clean['cardio']

mi_scores = mutual_info_classif(X, y, random_state=42)
mi_df = pd.DataFrame({'Feature': X.columns, 'MI Score': mi_scores})
mi_df = mi_df.sort_values(by='MI Score', ascending=False).reset_index(drop=True)
print(mi_df)
```

4.2.1 Feature Selection Analysis

Based on the Mutual Information (MI) scores, *Pulse Pressure*, *BMI*, and *Age-Cholesterol Risk* ranked as the top predictors. The *id* column was dropped entirely as it holds no predictive value. All newly engineered features were retained, as they significantly improved the decision boundaries compared to the baseline clinical variables.

4.3 SQLite Integration & Custom Queries

The cleaned dataset was uploaded to a local SQLite database (`data/cardio_cleaned.db`). We executed 4 specific analytical queries to validate clinical profiles:

```
import sqlite3
import pandas as pd

db_path = '../data/cardio_cleaned.db'
conn = sqlite3.connect(db_path)
df_clean.to_sql('cardio', conn, if_exists='replace', index=False)


q1 = """
SELECT active, alco, AVG(cardio) * 100 AS cvd_percentage, COUNT(*) AS patient_count
```

```
FROM cardio GROUP BY active, alco;
"""
print("Q1: Lifestyle Risk\n", pd.read_sql_query(q1, conn))

 (Patients > 50 Years Old)
q2 = """
SELECT cholesterol, AVG(pulse_pressure) AS avg_pulse_pressure, AVG(bmi) AS avg_bmi
FROM cardio WHERE (age / 365.25) > 50
GROUP BY cholesterol;
"""
print("\nQ2: Age & Cholesterol Risk\n", pd.read_sql_query(q2, conn))

(Undiagnosed but High Risk)
q3 = """
SELECT COUNT(*) AS hidden_danger_patients
FROM cardio
WHERE cardio = 0 AND cholesterol = 3 AND gluc = 3 AND bmi > 30;
"""
print("\nQ3: Hidden Danger Count\n", pd.read_sql_query(q3, conn))


q4 = """
SELECT gender, AVG(ap_hi) AS avg_systolic, AVG(weight) AS avg_weight
FROM cardio GROUP BY gender;
"""
print("\nQ4: Demographics\n", pd.read_sql_query(q4, conn))

conn.close()
```

5 Phase 4 & 5: Modeling & Performance Summary

A train/test split (80/20) was executed with a strict zero data leakage policy. We evaluated four models using robust preprocessor pipelines.

Table 3: Model Benchmarking Comparison

Model	Accuracy	Precision	Recall	F_1-score	ROC-AUC	**Training Time**
Tuned XGBoost	**73.57%**	**75.39%**	**70.21%**	**72.71%**	**80.12%**	**14.2s**
Logistic Regression	72.84%	74.92%	68.61%	71.63%	79.28%	0.8s
Random Forest	71.21%	72.10%	69.15%	70.59%	77.40%	5.6s
K-Nearest Neighbors	69.80%	70.35%	68.10%	69.21%	74.50%	2.1s

5.1 Model Training Code

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state
    =42, stratify=y)

num_features = ['age', 'height', 'weight', 'ap_hi', 'ap_lo', 'bmi', 'pulse_pressure', '
    age_cholesterol_risk']
cat_features = ['cholesterol', 'gluc']
bin_features = ['gender', 'smoke', 'alco', 'active']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(drop='first'), cat_features),
        ('bin', 'passthrough', bin_features)
    ]
```

```
)
```

5.2 Justification of F_1-Score

In healthcare, raw accuracy is a misleading metric. We must minimize False Negatives (patients with active CVD diagnosed as healthy, which could be fatal) and False Positives (wasting clinical stress-test resources on healthy patients). The F_1-score carefully balances Precision and Recall, protecting both the patient's long-term health and the clinic's operational resources.

6 Phase 6: Model Card & Ethical Considerations

- **Intended Use:** Decision-support tool for clinic triage.
- **Operational Limits:** Not validated for pediatric patients (< 18 years) or inputs outside human physiology.
- **Ethical Reflection:** Clinicians must oversee all automated recommendations (human-in-the-loop) and actively monitor for gender-related diagnostic disparities to ensure unbiased healthcare delivery.

7 Appendices

7.1 Appendix A: AI-Assisted Content Declaration

In accordance with the academic integrity guidelines of the AI & Data Science Basics module, generative AI (Google Gemini / Antigravity) was utilized exclusively as an educational support tool:

- **Code Generation:** AI was used to assist in structuring boilerplate Python pipelines and standardizing matplotlib visualizations. All logic regarding strict data leakage boundaries, train/test splits, and physical outlier thresholds was manually designed, verified, and audited by the team.
- **Text Drafting:** AI assisted in formatting the LaTeX report and refining the grammar of the Executive Summary. The problem statement formulation, EDA insights, and metric evaluation justifications represent our team's original analytical reasoning.

7.2 Appendix B: Individual Contribution Statement

As a two-person team, project responsibilities were evenly distributed, and both members are jointly accountable for the full technical submission:

- **Assadiki Zaid:** Led Phase 1 (Data Ingestion), Phase 2 (EDA, Physical Outlier Detection, and Visualization), and drafted the Algorithm Selection rationale. Configured the Git repository, managed version control workflows, and led the documentation of the final Model Card.
- **Eddhouir Mohammed:** Led Phase 3 (Feature Engineering & SQLite Management), Phase 5 (Model Training, Pipelines, & GridSearchCV Optimization), and constructed the analytical presentation slide deck for the oral defense.